



ESCOLA
SUPERIOR
DE TECNOLOGIA
E GESTÃO

BASE DE DADOS – RESUMOS DE ESTUDO

Contacto : Diogo Batista - 8230367

Defina os seguintes termos: Bases de Dados, Sistema de Gestão de Bases de Dados identificando os seus componentes, Metadados.

1. Base de Dados

Uma base de dados é um conjunto organizado de dados relacionados entre si, guardados de forma a facilitar o seu acesso, gestão e atualização.

2. Sistema de Gestão de Bases de Dados (SGBD)

Um SGBD é um software que permite criar, gerir e aceder a bases de dados.

Os seus componentes principais incluem:

- motor de armazenamento,
- linguagem de definição de dados (DDL),
- linguagem de manipulação de dados (DML),
- controlo de transações,
- controlo de acessos,
- interface com o utilizador.

3. Metadados

Metadados são dados que descrevem outros dados. Numa base de dados, indicam a estrutura das tabelas, tipos de campos, relações e restrições.

Descreve os 5 componentes do ambiente de um SGBD e mostre como elas se relacionam.

Os 5 componentes do ambiente de um SGBD são:

1. Hardware:

Inclui os dispositivos físicos (servidores, discos, redes) onde os dados são armazenados e processados.

2. Software:

Inclui o próprio SGBD, o sistema operativo e os programas que interagem com o SGBD.

3. Dados:

São os elementos centrais do sistema. Incluem os dados armazenados e os metadados que os descrevem.

4. Utilizadores:

Podem ser administradores de base de dados (DBA), programadores, ou utilizadores finais que consultam ou atualizam os dados.

5. **Procedimentos:**

São as regras e instruções que orientam o uso e funcionamento do SGBD, como cópias de segurança, permissões ou manutenção.

Descreva o conceito de independência de dados e a sua importância num ambiente de bases de dados.

Independência de Dados é a capacidade de modificar os níveis inferiores da base de dados (lógico ou físico) sem afetar os níveis superiores. Isto significa que os programas e aplicações que usam os dados não precisam de ser alterados quando há mudanças na estrutura ou no armazenamento dos dados.

Existem dois tipos:

- **Independência física:** mudanças na forma como os dados são armazenados (por exemplo, mudar o tipo de ficheiro ou o local de armazenamento) não afetam a estrutura lógica.
- **Independência lógica:** alterações na estrutura lógica (como adicionar uma tabela ou campo) não afetam os programas que usam os dados.

Importância:

Permite manter e evoluir o sistema de base de dados de forma mais flexível e económica, sem ter de reescrever aplicações sempre que há mudanças internas.

Descreva em que situações será preferível a abordagem de Sistemas de Ficheiros comparativamente à abordagem de Sistemas de Bases de Dados, tendo em atenção as principais desvantagens da utilização de um SGBD.

A abordagem de **Sistemas de Ficheiros** pode ser preferível em situações simples, onde os requisitos de gestão de dados são básicos. Alguns exemplos incluem:

- Aplicações pequenas ou pessoais, onde o volume de dados é reduzido;
- Sistemas que não exigem acesso simultâneo por vários utilizadores;

- Casos em que não é necessário garantir integridade, segurança ou controlo rigoroso dos dados;
- Quando os recursos computacionais são limitados (por exemplo, em dispositivos com pouca memória ou capacidade de processamento).

Isto deve-se às **desvantagens da utilização de um SGBD**, que incluem:

- Maior complexidade e necessidade de formação para operar o sistema;
- Consumo elevado de recursos (memória, armazenamento e CPU);
- Custo financeiro associado à instalação e manutenção do SGBD;
- Tempo de configuração inicial mais demorado.

Assim, para tarefas simples, onde estas funcionalidades avançadas não são necessárias, os **sistemas de ficheiros** podem ser mais eficientes e económicos.

Descreva as principais características de um Sistema BD e faça a comparação com os Sistemas Baseados em Ficheiros

Características principais de um Sistema de Base de Dados (BD):

1. **Independência de dados:** Permite alterar a estrutura ou armazenamento dos dados sem afetar os programas.
2. **Redução da redundância:** Os dados são centralizados, evitando duplicação.
3. **Integridade dos dados:** Existem regras que garantem a consistência e validade dos dados.
4. **Segurança e controlo de acessos:** Permite definir permissões de leitura e escrita para diferentes utilizadores.
5. **Acesso simultâneo por vários utilizadores:** Garante consistência mesmo com vários acessos ao mesmo tempo.
6. **Recuperação em caso de falha:** O sistema pode restaurar dados em caso de erro ou falha do sistema.
7. **Linguagens de consulta (como SQL):** Facilitam o acesso e manipulação dos dados

Comparação com Sistemas Baseados em Ficheiros:

- Nos **sistemas de ficheiros**, os dados são geridos por cada aplicação de forma separada, o que leva à duplicação e dificuldade em manter a integridade.
- Falta controlo de acessos e segurança centralizada.
- É difícil partilhar dados entre programas e utilizadores.
- Não existe independência de dados, ou seja, qualquer alteração obriga a modificar os programas.
- A recuperação de dados em caso de falha é limitada ou inexistente.

Descreva as 3 gerações de SGBD.

As **3 gerações de Sistemas de Gestão de Bases de Dados (SGBD)** referem-se à evolução dos modelos de dados utilizados ao longo do tempo:

1. Primeira Geração – Modelos Hierárquico e em Rede (anos 60–70):

Nesta fase, os dados eram organizados em estruturas rígidas.

- **Modelo hierárquico:** os dados eram organizados em forma de árvore, com relações de “pai-filho”.
- **Modelo em rede:** permitia ligações mais complexas entre dados, com múltiplos caminhos.

Ambos os modelos eram pouco flexíveis e difíceis de manter.

2. Segunda Geração – Modelo Relacional (anos 70–80 até hoje):

Introduzido por E. F. Codd, este modelo organiza os dados em tabelas (ou relações), facilitando a consulta e a manipulação com a linguagem SQL.

É o modelo mais usado atualmente, devido à sua flexibilidade, simplicidade e forte base teórica.

3. Terceira Geração – Modelos Orientado a Objetos e Objeto-Relacional (anos 90 até hoje):

Estes modelos integram conceitos da programação orientada a objetos, permitindo armazenar dados complexos (como imagens, vídeos ou estruturas

compostas).

São úteis em aplicações multimédia, científicas e de engenharia.

Cada geração melhorou a flexibilidade, a eficiência e a capacidade de representar dados complexos.

O que são sublinguagens de dados? Porque são importantes?

Sublinguagens de dados são subconjuntos de uma linguagem de base de dados (como o SQL) que servem para realizar diferentes tipos de operações sobre os dados.

As principais sublinguagens são:

1. **DDL (Data Definition Language):** usada para definir a estrutura da base de dados (tabelas, campos, índices, etc.).
Exemplo: CREATE, ALTER, DROP.
2. **DML (Data Manipulation Language):** usada para manipular os dados (inserir, atualizar, eliminar, consultar).
Exemplo: SELECT, INSERT, UPDATE, DELETE.
3. **DCL (Data Control Language):** usada para controlar os acessos e permissões dos utilizadores.
Exemplo: GRANT, REVOKE.
4. **TCL (Transaction Control Language):** usada para gerir transações e garantir a integridade dos dados.
Exemplo: COMMIT, ROLLBACK.

Importância:

As sublinguagens de dados são importantes porque permitem separar e organizar as diferentes tarefas num sistema de base de dados, facilitando a gestão, o controlo de segurança e a integridade dos dados. Além disso, tornam o uso do SGBD mais claro e eficiente.

Apresente as diferenças entre DDL e DML.
--

Diferenças entre DDL e DML:

1. **Função principal:**
 - **DDL (Data Definition Language):** serve para definir ou modificar a estrutura da base de dados, como tabelas, campos e índices.
 - **DML (Data Manipulation Language):** serve para manipular os dados dentro das tabelas, como inserir, consultar, atualizar ou apagar registos.
2. **Comandos típicos:**
 - **DDL:** CREATE, ALTER, DROP, TRUNCATE.
 - **DML:** SELECT, INSERT, UPDATE, DELETE.
3. **Efeito nos dados:**
 - **DDL:** altera a estrutura da base de dados.
 - **DML:** altera o conteúdo (os dados) da base de dados.
4. **Transações:**
 - **DDL:** normalmente não permite rollback (as alterações são definitivas).
 - **DML:** permite rollback e commit, pois opera dentro de transações.

Apresente as diferenças entre DML Procedimentais e DML Não Procedimentais.

Diferenças entre DML Procedimentais e DML Não Procedimentais:

1. **Forma de utilização:**
 - **DML Procedimentais:** o programador tem de indicar **como** os dados devem ser obtidos (passos detalhados).
 - **DML Não Procedimentais:** o programador indica **o que** pretende obter, sem descrever os passos.
2. **Exemplo de DML Procedimental:**
 - Linguagens como COBOL ou PL/SQL, onde é necessário percorrer os registos passo a passo (com laços, condições, etc.).
3. **Exemplo de DML Não Procedimental:**
 - SQL (em especial o comando SELECT), onde basta declarar os dados pretendidos, e o sistema trata da execução.
4. **Nível de complexidade:**
 - **Procedimental:** mais complexo e com maior controlo sobre o processo.
 - **Não Procedimental:** mais simples e mais próximo da linguagem natural.

Discuta a função de um System Catalog.

O **System Catalog** (ou **Catálogo do Sistema**) é um conjunto de tabelas e metadados mantido pelo SGBD, que descreve a própria base de dados.

Importância:

Sem o System Catalog, o SGBD não conseguiria interpretar nem gerir os dados, pois não teria informação sobre a estrutura nem regras do sistema. É, por isso, um componente essencial e central.

Explique o conceito de Database Schema e descreva os três tipos de schema numa BD.

Conceito de Database Schema:

Um **Database Schema** (ou **esquema de base de dados**) é a descrição formal da estrutura de uma base de dados. Define como os dados estão organizados, quais as tabelas existentes, os campos de cada tabela, os tipos de dados, as relações entre tabelas e as restrições aplicadas (como chaves primárias e estrangeiras).

Três tipos de schema numa base de dados:

1. **Esquema Externo (ou de Visão):**

Representa a forma como cada utilizador ou aplicação vê os dados. Cada utilizador pode ter uma visão personalizada da base de dados, com acesso apenas a certas partes da informação.

2. **Esquema Conceptual:**

Define a estrutura lógica da base de dados para toda a organização. Inclui todas as entidades, atributos, relações e restrições, mas sem se preocupar com detalhes de armazenamento físico.

3. **Esquema Interno (ou Físico):**

Descreve a forma como os dados são armazenados fisicamente no sistema. Inclui informações sobre ficheiros, índices, estruturas de armazenamento e caminhos de acesso.

Discuta a função e importância do modelo conceptual.
--

Função do modelo conceptual:

O **modelo conceptual** representa a estrutura lógica dos dados de forma abstrata, ou seja, sem considerar como os dados serão armazenados fisicamente. A sua função é identificar e descrever as **entidades**, **atributos** e **relações** relevantes no contexto do sistema de informação.

Importância do modelo conceptual:**1. Compreensão clara dos dados:**

Ajuda a representar o que realmente é necessário armazenar, com base nos requisitos dos utilizadores.

2. Comunicação entre técnicos e utilizadores:

Serve como um meio de comunicação entre quem conhece o funcionamento do negócio e quem vai implementar o sistema.

3. Base para os modelos seguintes:

O modelo conceptual é o ponto de partida para criar o modelo lógico e, depois, o modelo físico da base de dados.

4. Evita erros na fase de implementação:

Permite detetar falhas, redundâncias ou omissões antes da base de dados ser criada, o que poupa tempo e custos.

Resumo:

O modelo conceptual é fundamental para garantir que a base de dados responde corretamente às necessidades da organização, sendo o primeiro passo no desenvolvimento de um sistema bem estruturado.

Identifique quais os objetivos da arquitetura de três níveis ANSI/SPARC.

Objetivos da arquitetura de três níveis ANSI/SPARC:

A arquitetura de três níveis proposta pela ANSI/SPARC tem como principais objetivos:

1. **Separar os diferentes níveis de abstração dos dados:**
Divide o sistema de base de dados em três níveis — **interno**, **conceptual** e **externo** — permitindo uma melhor organização e compreensão da estrutura da base de dados.
2. **Garantir a independência dos dados:**
 - **Independência física:** alterações no nível interno (forma de armazenamento) não afetam o nível conceptual.
 - **Independência lógica:** alterações no nível conceptual (estrutura dos dados) não afetam as aplicações no nível externo.
3. **Facilitar a segurança e controlo de acessos:**
Permite definir diferentes visões (nível externo) para diferentes utilizadores, restringindo o acesso apenas aos dados necessários.
4. **Melhorar a flexibilidade e manutenção:**
Torna mais fácil adaptar o sistema a mudanças nos requisitos, sem necessidade de alterar todas as partes do sistema.

Resumo:

A arquitetura ANSI/SPARC foi criada para tornar os sistemas de base de dados mais modulares, seguros, flexíveis e fáceis de manter, garantindo separação entre a forma como os dados são armazenados, estruturados e utilizados.

A arquitetura ANSI/SPARC identifica três níveis nos SGBD. Descreva pormenorizadamente o nível intermédio, identificando o seu nome, e o que se pretende que este nível represente.

A arquitetura ANSI/SPARC define três níveis nos sistemas de gestão de bases de dados (SGBD): **nível interno**, **nível conceptual** e **nível externo**.

Nível intermédio: Nível Conceptual

O **nível conceptual** é o nível intermédio da arquitetura e representa a **visão global e lógica da base de dados** para toda a organização, independentemente de como os dados são armazenados fisicamente (nível interno) ou apresentados aos utilizadores (nível externo).

O que representa este nível:

- **Estrutura lógica da base de dados:** descreve todas as entidades, atributos e relações existentes no sistema.
 - **Regras de integridade:** define as restrições e dependências entre dados, como chaves primárias e estrangeiras.
 - **Segurança e permissões globais:** define regras de acesso aos dados a nível geral, sem entrar nos detalhes individuais de cada utilizador.
 - **Independência lógica dos dados:** permite modificar a estrutura conceptual sem afetar as aplicações que acedem aos dados (desde que a visão externa se mantenha).
-

Resumo:

O **nível conceptual** é fundamental porque atua como intermediário entre o armazenamento físico dos dados e a forma como são apresentados aos utilizadores. Garante uma visão consistente e integrada da informação dentro do SGBD, assegurando a independência lógica dos dados.

Identifique quais as propriedades das Relações no Modelo Relacional.

Propriedades das Relações no Modelo Relacional:

No modelo relacional, os dados são organizados em **relações**, que podem ser representadas como **tabelas**. Cada relação possui as seguintes propriedades fundamentais:

1. Nome único:

Cada relação (tabela) tem um nome distinto dentro da base de dados.

2. **Tuplos distintos:**

Não existem tuplos (linhas/registos) repetidos numa relação. Cada linha é única.

3. **Sem ordem entre os tuplos:**

A ordem das linhas numa tabela não tem significado. Pode variar sem alterar o conteúdo da relação.

4. **Sem ordem entre os atributos:**

A ordem das colunas também não influencia a relação, desde que os nomes e os dados correspondam corretamente.

5. **Atributos atômicos:**

Cada valor numa célula da tabela deve ser indivisível (atômico), ou seja, não pode conter múltiplos valores ou listas.

6. **Cada atributo tem um nome único:**

Os nomes das colunas (atributos) dentro da mesma tabela devem ser distintos.

Resumo:

Estas propriedades garantem a **simplicidade**, a **consistência** e a **flexibilidade** do modelo relacional, facilitando a manipulação e a consulta dos dados através de linguagens como o SQL.

Defina as duas principais regras de integridade no Modelo Relacional.

As duas principais regras de integridade no Modelo Relacional são:

1. **Integridade da entidade:**

Garante que cada tuplo (linha) de uma relação pode ser **unicamente identificado**.

Para isso, cada tabela deve ter uma **chave primária** (primary key), cujos valores:

- Não podem ser **nulos** (NULL);
- Devem ser **únicos** (sem repetições).

Exemplo: Uma tabela de clientes pode ter o campo “ID_Cliente” como chave primária.

2. Integridade referencial:

Garante a **coerência entre dados de diferentes tabelas** que se relacionam entre si.

Quando uma tabela contém uma **chave estrangeira** (foreign key), essa chave deve:

- Corresponder a um valor existente na chave primária da outra tabela;
- Ou ser nula, se a relação o permitir.

Exemplo: A chave estrangeira “ID_Cliente” numa tabela de encomendas deve corresponder a um cliente que existe na tabela de clientes.

Resumo:

Estas regras garantem a **consistência e validade** dos dados no modelo relacional, evitando erros como dados repetidos, órfãos ou inválidos.

Na apresentação do modelo relacional foram enunciados alguns conceitos. Explique o que significa cada um dos seguintes termos: Relação, Atributo, Domínio, Tuplo, Grau, Cardinalidade.

1. Relação:

No modelo relacional, uma **relação** representa uma **tabela** que armazena dados. Cada relação tem um nome único e é composta por linhas (tuplos) e colunas (atributos).

2. Atributo:

Um **atributo** é uma **coluna** de uma tabela. Representa uma característica dos dados armazenados, como por exemplo “Nome” ou “Idade”.

3. Domínio:

O **domínio** de um atributo é o **conjunto de valores válidos** que esse atributo pode assumir.

Exemplo: O domínio do atributo “Idade” pode ser o conjunto dos números inteiros positivos.

4. Tuplo:

Um **tuplo** (ou tupla) é uma **linha** da tabela. Representa um registo completo, ou seja, uma ocorrência concreta da relação.

Exemplo: Um cliente específico com ID, nome e idade.

5. Grau:

O **grau** de uma relação é o **número de atributos** (colunas) que ela possui.

Exemplo: Uma tabela com 4 colunas tem grau 4.

6. Cardinalidade:

A **cardinalidade** de uma relação é o **número de tuplos** (linhas) existentes na tabela.

Exemplo: Se uma tabela tem 10 linhas, a sua cardinalidade é 10.

Defina as cinco operações principais de álgebra relacional. Defina a Junção, Intersecção, Divisão através da utilização das cinco operações básicas.

As cinco operações principais da Álgebra Relacional:

1. Seleção (σ):

Seleciona as **linhas (tuplos)** de uma relação que satisfazem uma condição específica.

Exemplo: $\sigma_{\text{idade} > 30}(\text{Pessoa})$ — devolve todas as pessoas com mais de 30 anos.

2. Projeção (π):

Seleciona **colunas (atributos)** específicas de uma relação, eliminando duplicados.

Exemplo: $\pi_{\text{nome, idade}}(\text{Pessoa})$ — devolve apenas os nomes e idades.

3. União ($\cup$):

Combina os tuplos de duas relações compatíveis (mesmos atributos).

Exemplo: $A \cup B$ — devolve os tuplos que estão em A, em B, ou em ambos.

4. Diferença (-):

Retorna os tuplos que existem numa relação e **não existem** na outra.

Exemplo: $A - B$ — devolve os tuplos que estão em A mas não em B.

5. Produto Cartesiano (*):

Combina **todos os tuplos de uma relação com todos os da outra**.

Exemplo: $A \times B$ — se A tem 3 tuplos e B tem 2, o resultado terá 6.

Operações Derivadas:

As operações abaixo podem ser construídas a partir das cinco operações básicas:

1. Junção ($\bowtie$):

Combina tuplos de duas relações com base numa condição comum (semelhante ao produto cartesiano seguido de seleção).

Exemplo: $A \bowtie_{A.id = B.id} B \equiv \sigma_{A.id = B.id}(A \times B)$

2. Interseção ($\cap$):

Retorna apenas os tuplos que estão **em ambas** as relações.

Exemplo: $A \cap B \equiv A - (A - B)$

3. Divisão ($\div$):

Usada para encontrar tuplos numa relação que estão relacionados com **todos** os tuplos de outra.

É uma operação mais complexa, mas pode ser construída com as operações básicas:

Exemplo: Se $A(X, Y)$ e $B(Y)$, então:

$A \div B = \pi_{X}(A) - \pi_{X}((\pi_{X}(A) \times B) - A)$

Resumo:

A Álgebra Relacional fornece um conjunto de operações para manipular e consultar dados de forma matemática e precisa. As cinco operações básicas (seleção, projeção, união, diferença, produto cartesiano) são suficientes para construir operações mais complexas como junção, interseção e divisão.

Enuncie a definição de cada um dos seguintes termos: Chave Candidata, Chave Primária, Chave Estrangeira.

1. Chave Candidata:

Uma **chave candidata** é um conjunto mínimo de atributos que **identifica univocamente cada tuplo** (linha) numa relação (tabela).

Pode haver várias chaves candidatas numa tabela, mas nenhuma deve conter atributos desnecessários (são minimais).

2. Chave Primária:

A **chave primária** é a chave candidata **escolhida** para identificar univocamente cada tuplo da tabela.

- Os valores da chave primária **não podem repetir-se**;
 - **Não pode conter valores nulos (NULL)**.
-

3. Chave Estrangeira:

Uma **chave estrangeira** é um atributo (ou conjunto de atributos) numa tabela que **faz referência à chave primária de outra tabela**.

É usada para estabelecer relações entre tabelas e garantir a **integridade referencial** dos dados.

Resumo:

- A **chave candidata** é qualquer possível identificador único;
- A **chave primária** é a chave candidata escolhida para representar a tabela;
- A **chave estrangeira** liga tabelas entre si, apontando para uma chave primária noutra tabela.

Descreva os tipos de anomalias de atualização (dê exemplos) que podem ocorrer numa relação que contém dados redundantes.

1. Anomalia de Inserção:

Ocorre quando não é possível **inserir certos dados porque falta outra informação redundante**.

Exemplo:

Numa tabela que junta *Estudantes* e *Disciplinas*, se quisermos registar um novo estudante que ainda não está inscrito em nenhuma disciplina, não o podemos inserir sem inventar uma disciplina falsa, pois o campo “Disciplina” não pode ficar vazio.

2. Anomalia de Eliminação:

Acontece quando ao eliminar um registo, **perdemos informação importante que estava repetida**.

Exemplo:

Se eliminarmos o único registo do aluno João na disciplina de Matemática, também apagamos todos os dados sobre a disciplina “Matemática”, mesmo que ainda nos interessem.

3. Anomalia de Atualização:

Surge quando **a mesma informação aparece em vários locais** e uma atualização não é feita em todos, gerando inconsistência.

Exemplo:

Se o nome de um professor estiver repetido em várias linhas e for atualizado numa só, ficamos com versões diferentes do mesmo professor.

Resumo:

Estas anomalias dificultam a manutenção dos dados e comprometem a **consistência, integridade e fiabilidade** da base de dados. A melhor forma de evitá-las é através da **normalização** do esquema da base de dados.

Explique o que significam cada uma das cláusulas utilizadas no comando SELECT.
--

O comando **SELECT** é usado em SQL para **consultar dados** numa base de dados relacional. Este comando pode incluir várias **cláusulas**, cada uma com uma função específica.

1. SELECT

Indica **quais os atributos (colunas)** a serem devolvidos na consulta.

Exemplo: SELECT nome, idade

```
SELECT nome, idade
```

```
FROM Alunos;
```

Devolve os nomes e idades de todos os alunos.

2. FROM

Indica **de que tabela(s)** os dados devem ser obtidos.

Exemplo: FROM Alunos

```
SELECT nome
```

```
FROM Professores;
```

Vai buscar os nomes da tabela “Professores”.

3. WHERE

Aplica uma **condição** para filtrar os tuplos (linhas) devolvidos.

Exemplo: WHERE idade > 18

```
SELECT nome, idade
```

```
FROM Alunos
```

```
WHERE idade > 18;
```

Devolve apenas os alunos com mais de 18 anos.

4. GROUP BY

Agrupa os resultados com base em um ou mais atributos, normalmente para usar funções agregadas (como SUM, AVG, COUNT).

Exemplo: GROUP BY curso

```
SELECT curso, COUNT(*)  
  
FROM Alunos  
  
GROUP BY curso;
```

Mostra o número de alunos por curso.

5. HAVING

Filtra os **grupos** criados pela cláusula GROUP BY, de forma semelhante à WHERE, mas aplicado aos grupos.

Exemplo: HAVING COUNT(*) > 5

```
SELECT curso, COUNT(*)  
  
FROM Alunos  
  
GROUP BY curso  
  
HAVING COUNT(*) > 10;
```

Mostra apenas os cursos com mais de 10 alunos.

6. ORDER BY

Ordena os resultados por uma ou mais colunas, em **ordem crescente (ASC)** ou **decrescente (DESC)**.

Exemplo: ORDER BY nome ASC

```
SELECT nome, idade  
  
FROM Alunos  
  
ORDER BY idade DESC;
```

Lista os alunos por ordem decrescente de idade.

Explique a importância e aplicação da cláusula WHERE em UPDATE e DELETE.

A cláusula **WHERE** é **essencial** nas instruções **UPDATE** e **DELETE**, pois define **quais os registos (tuplos) que devem ser alterados ou eliminados**. Sem esta cláusula, a instrução afeta **todos os registos** da tabela, o que pode causar **perda de dados ou alterações indesejadas**.

1. Aplicação em UPDATE:

Serve para **atualizar apenas os registos que satisfazem uma condição**.

Exemplo:

```
UPDATE Alunos  
  
SET curso = 'Engenharia Informática'  
  
WHERE id = 101;
```

Atualiza o curso apenas do aluno com ID 101.

Sem WHERE:

```
UPDATE Alunos  
  
SET curso = 'Engenharia Informática';
```

Iria alterar o curso de **todos os alunos**, o que normalmente não é desejado.

2. Aplicação em DELETE:

Serve para **eliminar apenas os registos que satisfazem uma condição.**

Exemplo:

```
DELETE FROM Alunos
```

```
WHERE idade < 18;
```

Elimina apenas os alunos com menos de 18 anos.

Sem WHERE:

```
DELETE FROM Alunos;
```

Elimina **todos os registos da tabela**, o que pode levar à perda total de dados.

Explique como funciona a cláusula GROUP BY. Qual a diferença entre a cláusula WHERE e HAVING?

Cláusula GROUP BY – Como funciona:

A cláusula **GROUP BY** é usada em instruções SELECT para **agrupar os registos** da tabela com base nos valores de um ou mais atributos (colunas). É especialmente útil quando se usam **funções de agregação** como COUNT(), SUM(), AVG(), MAX() e MIN().

Exemplo:

```
SELECT curso, COUNT(*)
```

```
FROM Alunos
```

```
GROUP BY curso;
```

Este comando conta quantos alunos há em cada curso.

Cláusula Quando é usada Atua sobre

WHERE Antes do GROUP BY Filtra **linhas individuais** (registos da tabela)

HAVING Depois do GROUP BY Filtra **grupos de linhas** (após agregação)

Exemplo com ambas:

SELECT curso, COUNT(*)

FROM Alunos

WHERE idade > 18

GROUP BY curso

HAVING COUNT(*) > 5;

WHERE idade > 18: filtra os alunos com mais de 18 anos **antes** de agrupar.

HAVING COUNT(*) > 5: filtra os cursos que **têm mais de 5 alunos após** o agrupamento.

Resumo:

- GROUP BY serve para **agrupar registos** com o mesmo valor numa ou mais colunas.
- WHERE filtra **registos individuais** antes da agregação.
- HAVING filtra os **grupos resultantes** depois da agregação.

Quais as restrições aplicadas ao uso de funções de agregação no comando SELECT? De que forma os valores nulos (NULL) afetam as funções de agregação?

Restrições ao uso de funções de agregação no comando SELECT:

1. **Não podem ser usadas diretamente na cláusula WHERE:**

As funções de agregação (como COUNT, SUM, AVG, MAX, MIN) não podem aparecer na cláusula WHERE, pois esta atua **antes da agregação**.

→ Para filtrar com base no resultado da agregação, deve-se usar a cláusula **HAVING**.

2. **Devem ser usadas com GROUP BY quando se misturam com colunas não agregadas:**

Se usares uma função de agregação com outras colunas no SELECT, **as colunas não agregadas devem aparecer na cláusula GROUP BY**.

→ Caso contrário, dá erro.

Exemplo correto:

```
SELECT curso, COUNT(*)  
FROM Alunos  
GROUP BY curso;
```

Efeito dos valores NULL nas funções de agregação:

1. **COUNT(*):** Conta **todas as linhas**, incluindo as que contêm valores NULL.
2. **COUNT(campo):** Conta apenas as linhas onde **o campo não é NULL**.
3. **SUM(campo), AVG(campo), MAX(campo), MIN(campo):**
 - Ignoram os valores **NULL** ao calcular o resultado.
 - Só consideram os valores reais.

Exemplo prático:

Tabela com os valores: 10, 20, NULL, 30

- COUNT(*) = 4
- COUNT(valor) = 3
- SUM(valor) = 60
- AVG(valor) = 20
- MAX(valor) = 30
- MIN(valor) = 10

Resumo:

As funções de agregação têm restrições no seu uso com SELECT, especialmente com WHERE e GROUP BY, e os valores NULL **não são considerados nos cálculos**, exceto em COUNT(*).

Explique o que entende por Integridade Referencial.
Identifique e descreva quais as ações que se podem utilizar nas subcláusulas ON DELETE e ON UPDATE.

Integridade Referencial – Definição:

A **integridade referencial** é uma regra do modelo relacional que **garante a consistência das ligações entre tabelas**.

Esta integridade assegura que os valores de uma **chave estrangeira (foreign key)** em uma tabela **correspondem a valores existentes** na **chave primária** da tabela relacionada.

Exemplo simples:

Se a tabela Encomendas tiver uma coluna ID_Cliente como chave estrangeira, este valor **deve existir** na tabela Clientes como ID.

Subcláusulas ON DELETE e ON UPDATE:

Estas subcláusulas definem **o que deve acontecer com os registos dependentes** quando há alterações na tabela referenciada.

Ações possíveis:

1. CASCADE

– Se um registo na tabela principal for **apagado (DELETE)** ou **alterado (UPDATE)**, a alteração é **propagada automaticamente** para a tabela dependente.

Exemplo: apagar um cliente → todas as suas encomendas são também apagadas.

2. SET NULL

– Quando há DELETE ou UPDATE, o valor da chave estrangeira na tabela dependente é definido como **NULL**.

– Só funciona se o campo aceitar valores nulos.

Exemplo: se o cliente for eliminado, o campo ID_Cliente na tabela Encomendas passa a NULL.

3. SET DEFAULT

– A chave estrangeira na tabela dependente é substituída pelo **valor por defeito (DEFAULT)** definido para o campo.

– Requer que o atributo tenha um valor DEFAULT especificado.

4. NO ACTION / RESTRICT

– **Não permite a alteração** se houver registos dependentes.

– Gera um erro, impedindo a operação que viola a integridade referencial.

Exemplo: não se pode apagar um cliente que tem encomendas associadas.

Resumo:

A integridade referencial **mantém as ligações corretas entre tabelas**. As subcláusulas ON DELETE e ON UPDATE permitem definir o **comportamento automático** quando há alterações nos dados referenciados, garantindo a **consistência e coerência** da base de dados.

Qual a diferença entre uma subquery e uma junção? Em que situações não é possível usar uma subquery?

1. Subquery (subconsulta):

Uma **subquery** é uma instrução SELECT colocada **dentro de outra instrução SQL** (geralmente dentro de SELECT, FROM, WHERE ou HAVING).

Ela **devolve um valor ou conjunto de valores**, que depois é utilizado pela consulta principal.

Exemplo:

```
SELECT nome
FROM Alunos
WHERE idade = (SELECT MAX(idade) FROM Alunos);
```

2. Junção (JOIN):

Uma **junção** combina **dados de duas ou mais tabelas** com base numa condição comum (normalmente uma chave estrangeira que liga duas tabelas).

É usada para **mostrar colunas de várias tabelas ao mesmo tempo**.

Exemplo:

```
SELECT Alunos.nome, Cursos.nome
FROM Alunos
JOIN Cursos ON Alunos.codCurso = Cursos.codCurso;
```

Situações em que não é possível usar subquery:

- Quando a lógica **requer múltiplas colunas** de tabelas diferentes na mesma linha → uma **junção** é obrigatória.
Exemplo: Mostrar o nome do aluno e o nome do curso. Uma subquery não consegue fazer isso diretamente.

- Quando a subquery devolve mais de um valor onde só é esperado **um único valor escalar**.

Exemplo incorreto:

```
SELECT nome
FROM Alunos
WHERE idade = (SELECT idade FROM Alunos);
```

Dá erro se a subquery devolver várias idades.

Resumo:

- Usa **subquery** quando precisas de **um valor específico** ou queres **filtrar** com base noutros dados.
- Usa **junção** quando precisas de **combinar dados de várias tabelas lado a lado**.
- As subqueries têm limitações quando precisam de devolver múltiplos valores para uso direto.

Diferencia entre os 3 tipos de subqueries.

1. Subquery Escalar (Scalar Subquery):

- **Devolve apenas um único valor** (um único campo e uma única linha).
- Pode ser usada onde seria esperado um valor simples, como numa condição ou atribuição.

Exemplo:

```
SELECT nome
FROM Alunos
WHERE idade = (SELECT MAX(idade) FROM Alunos);
```

Uso: Comparar um valor com o resultado da subquery.

2. Subquery de Linha ou Tabela (Row/Table Subquery):

- Pode devolver várias colunas e/ou várias linhas.
- Usada normalmente no FROM, como se fosse uma tabela temporária.

Exemplo:

```
SELECT media.idCurso, media.mediaIdade
FROM (
    SELECT idCurso, AVG(idade) AS mediaIdade
    FROM Alunos
    GROUP BY idCurso
) AS media;
```

Uso: Criar uma tabela derivada com base numa subconsulta.

3. Subquery de Conjunto (Set Subquery):

- Devolve várias linhas (uma coluna).
- É usada com operadores como:
 - IN
 - ANY / SOME
 - ALL
 - EXISTS

Exemplo com IN:

```
SELECT nome
FROM Alunos
WHERE idCurso IN (SELECT idCurso FROM Cursos WHERE area = 'Informática');
```

Uso: Verificar se um valor está presente num **conjunto de resultados**.

Tipo de Subquery	Devolve	Usado em...	Exemplo de uso
Escalar	1 valor (1 linha, 1 coluna)	WHERE, SELECT, etc.	Comparação com =
De Linha/Tabela	Múltiplas linhas/colunas	FROM	Subtabela temporária
De Conjunto	1 coluna, várias linhas	WHERE com IN, ANY, etc.	Presença num conjunto

Descreva as diferenças entre as cinco operações de junção: Theta Join, Equijoin, Natural Join, Outer Join e Semijoin. Dê exemplos para suportar a sua resposta.

No modelo relacional, as **junções** são usadas para **combinar dados de duas ou mais tabelas** com base em condições de relação. Eis os cinco principais tipos:

1. Theta Join (θ-Join):

- É o tipo mais **geral** de junção.
- Usa **qualquer operador de comparação**: =, >, <, >=, <=, !=.
- A condição é definida explicitamente no ON ou WHERE.

Exemplo:

```
SELECT *
FROM Funcionarios F
JOIN Departamentos D
ON F.salario > D.orcamento;
```

Devolve funcionários com salário superior ao orçamento do departamento.

2. Equijoin:

- É um **caso especial de Theta Join** em que a condição de junção usa **apenas o operador de igualdade (=)**.
- Não remove colunas duplicadas.

Exemplo:

```
SELECT *  
FROM Funcionarios F  
JOIN Departamentos D  
ON F.codDept = D.codDept;
```

Junta as duas tabelas onde os códigos de departamento coincidem.

3. Natural Join:

- Também baseada na igualdade (=), mas é **automática**:
 - Usa **todos os atributos com o mesmo nome** em ambas as tabelas.
 - **Remove as colunas duplicadas** no resultado.

Exemplo:

```
SELECT *  
FROM Funcionarios  
NATURAL JOIN Departamentos;
```

Junção feita automaticamente pela coluna com nome comum, como codDept.

4. Outer Join:

- Inclui também os **registos que não têm correspondência** na outra tabela.
- Três tipos:
 - **Left Outer Join**: mantém todos os registos da tabela da esquerda.
 - **Right Outer Join**: mantém os da direita.

- **Full Outer Join:** mantém todos os registos de ambas, com NULL onde não há correspondência.

Exemplo (Left):

```
SELECT *  
FROM Funcionarios F  
LEFT JOIN Departamentos D  
ON F.codDept = D.codDept;
```

Devolve todos os funcionários, mesmo os que não têm departamento.

5. Semijoin (Semi Join):

- Não está diretamente disponível em SQL, mas pode ser **simulado com subqueries**.
- **Retorna apenas os registos da primeira tabela** que têm correspondência na segunda, **sem combinar os dados** da segunda tabela.

Exemplo (simulado com EXISTS):

```
SELECT nome  
FROM Funcionarios F  
WHERE EXISTS (  
    SELECT * FROM Departamentos D  
    WHERE F.codDept = D.codDept  
);
```

Lista nomes dos funcionários que estão em algum departamento.

Tipo de Junção	Operador usado	Inclui dados sem correspondência?	Remove duplicados?	Combina colunas?
Theta Join	Qualquer operador	Não	Não	Sim
Equijoin	Igualdade (=)	Não	Não	Sim
Natural Join	Igualdade automática	Não	Sim	Sim
Outer Join	Igualdade (=)	Sim (depende do tipo)	Não	Sim
Semijoin	Igualdade (via subquery)	Não (só da 1. ^a tabela)	–	Não

O que são Triggers de bases de dados e para que servem?

O que são Triggers numa base de dados?

Uma **trigger** (ou **gatilho**) é um **objeto do sistema de gestão de bases de dados** que contém **um conjunto de instruções SQL** e que é **executado automaticamente** quando ocorre um determinado **evento** numa tabela.

Para que servem as Triggers?

As triggers servem para:

1. **Executar ações automáticas** quando se fazem operações INSERT, UPDATE ou DELETE numa tabela;
2. **Aplicar regras de negócio** (por exemplo, impedir alterações fora de horário);
3. **Garantir a integridade dos dados** (complementarmente às restrições do SGBD);
4. **Gerar logs ou auditorias** (ex: guardar alterações feitas a um registo);
5. **Atualizar outras tabelas** de forma automática (ex: atualizar o stock depois de uma venda).

Exemplo simples:

```
CREATE TRIGGER atualizaStock
AFTER INSERT ON Encomendas
FOR EACH ROW
BEGIN
    UPDATE Artigos
    SET stock = stock - NEW.quantidade
    WHERE codArtigo = NEW.codArtigo;
END;
```

Esta trigger é ativada automaticamente após uma nova encomenda, atualizando o stock do artigo correspondente.

Resumo:

Triggers são **automatismos no SGBD** que **reagem a eventos** nas tabelas, permitindo **automatizar processos, proteger os dados e garantir regras de negócio** sem intervenção manual.

Discuta as vantagens e desvantagens dos Triggers
--

Vantagens das Triggers:

1. **Automatização de tarefas:**

Permitem que certas ações sejam executadas **automaticamente**, sem intervenção do utilizador (ex: atualizar stock, criar registos de auditoria).

2. **Reforço da integridade dos dados:**

Ajudam a **garantir regras de negócio** que não são facilmente expressas com restrições padrão (como CHECK, FOREIGN KEY, etc.).

3. **Centralização da lógica:**

O código fica **dentro da base de dados**, o que evita repetições em várias aplicações diferentes e assegura consistência.

4. **Melhoria na segurança e controlo:**

Podem ser usadas para **impedir alterações indesejadas** ou para registar todas as modificações feitas aos dados.

Desvantagens das Triggers:

1. Dificuldade de manutenção:

O código das triggers pode ser **difícil de ler e de localizar**, especialmente em sistemas grandes com muitas triggers ocultas.

2. Desempenho:

Triggers **podem reduzir a performance**, pois adicionam operações extra sempre que são ativadas (ex: ao inserir um registo, faz-se mais do que apenas guardar dados).

3. Execução implícita:

Como são executadas automaticamente, **podem ocorrer efeitos indesejados** sem que o utilizador perceba, dificultando o controlo de erros.

4. Ordem de execução incerta:

Quando há múltiplas triggers para o mesmo evento, a **ordem de execução nem sempre é garantida**, o que pode causar comportamentos inesperados.

Resumo:

As **triggers** são úteis para **automatizar tarefas e garantir regras de integridade**, mas devem ser usadas com cuidado, pois podem **complicar a manutenção** do sistema e **afetar o desempenho** da base de dados.

Qual a diferença entre Triggers Before, After, Instead Of.
--

Diferença entre Triggers BEFORE, AFTER e INSTEAD OF:

As **triggers** (ou gatilhos) são ativadas automaticamente quando ocorrem eventos como INSERT, UPDATE ou DELETE. A diferença entre os tipos está no **momento em que são executadas** e no **efeito que têm** sobre a operação.

1. BEFORE TRIGGER (Antes da operação)

- Executa-se **antes** da operação (INSERT, UPDATE ou DELETE) ser realizada.
- É usada para **validar ou modificar dados antes de serem inseridos ou alterados**.

- Pode **anular** a operação se for detetado um erro.

Exemplo de uso:

Verificar se um valor é válido antes de inserir ou impedir alterações fora de horário.

2. AFTER TRIGGER (Depois da operação)

- Executa-se **depois** da operação ter sido concluída com sucesso.
- É usada para **ações complementares**, como atualizar outras tabelas ou guardar registos de auditoria.

Exemplo de uso:

Após a inserção de uma encomenda, atualizar o stock do artigo.

3. INSTEAD OF TRIGGER (Em vez da operação)

- **Substitui** a operação original.
- É normalmente usada em **views (visões)**, onde não é possível modificar diretamente os dados.
- Permite **definir o que deve acontecer em vez da ação solicitada**.

Exemplo de uso:

Ao tentar atualizar uma view, a trigger INSTEAD OF redireciona a atualização para as tabelas subjacentes.

Tipo de Trigger	Quando é executada	Aplicação típica
BEFORE	Antes da operação	Validação ou alteração de dados
AFTER	Depois da operação	Ações complementares ou registos
INSTEAD OF	Em vez da operação	Modificação de views ou controlo total da ação

O que é uma transação? Dê exemplos.

O que é uma Transação numa base de dados?

Uma **transação** é um conjunto de operações SQL que são tratadas como uma **unidade lógica de trabalho**.

Ou seja, **ou todas as operações são executadas com sucesso**, ou **nenhuma é aplicada**.

Isto garante que a base de dados **permanece consistente**, mesmo em caso de falhas ou erros.

Características de uma transação (propriedades ACID):

1. **Atomicidade:** Tudo ou nada — a transação é executada por completo ou revertida.
2. **Consistência:** A base de dados passa de um estado consistente para outro.
3. **Isolamento:** Transações simultâneas não interferem entre si.
4. **Durabilidade:** Após ser confirmada, a transação mantém-se, mesmo em caso de falha no sistema.

Exemplo 1 – Transferência bancária:

Supondo que um cliente transfere 100€ da conta A para a conta B:

```
BEGIN;  
  
UPDATE Contas SET saldo = saldo - 100 WHERE conta_id = 'A';  
UPDATE Contas SET saldo = saldo + 100 WHERE conta_id = 'B';  
  
COMMIT;
```

Se uma das operações falhar, deve-se fazer:

ROLLBACK

Isto evita que o dinheiro desapareça ou que fique duplicado.

Exemplo 2 – Inserir encomenda e linhas:

```
BEGIN;

INSERT INTO Encomendas (numEncomenda, data, codCliente) VALUES ('E001', CURRENT_DATE, 'C123');

INSERT INTO LinhasEncomenda (numEncomenda, codArtigo, quantidade)
VALUES ('E001', 'A10', 2),
       ('E001', 'A20', 1);

COMMIT;
```

Se houver erro ao inserir uma das linhas, a encomenda inteira é anulada com ROLLBACK.

Resumo:

Uma transação é essencial para garantir que as operações na base de dados são realizadas de forma **segura, consistente e controlada**, especialmente em sistemas críticos como bancos, lojas online ou gestão de stocks.

Qual a diferença entre procedimento e uma função?

Diferença entre Procedimento e Função numa base de dados:

Tanto os **procedimentos (procedures)** como as **funções (functions)** são blocos de código armazenados no sistema de base de dados e usados para **executar tarefas automaticamente**. No entanto, existem diferenças importantes entre os dois.

1. Função (Function):

- **Devolve sempre um valor único** (obrigatório).
- Pode ser usada **dentro de expressões SQL**, como em SELECT, WHERE, CASE, etc.
- **Não altera dados diretamente** (em regra) — deve ser **determinística** e sem efeitos colaterais.

Exemplo:

```
CREATE FUNCTION calcular_iva(preco NUMERIC)
RETURNS NUMERIC AS $$
BEGIN
    RETURN preco * 0.23;
END;
```

```
-- Uso:
SELECT nome, calcular_iva(preco) FROM Artigos;
```

2. Procedimento (Procedure):

- Pode ou não devolver valores.
- Serve para **executar ações complexas**, como atualizações, inserções, loops, transações, etc.
- **Não pode ser usada num SELECT** — é chamada com CALL.
- Aceita **parâmetros de entrada e saída**.

Exemplo:

```
CREATE PROCEDURE atualizar_stock(cod_artigo TEXT, qtd INT)
AS $$
BEGIN
    UPDATE Artigos
    SET stock = stock - qtd
    WHERE codigo = cod_artigo;
END;
$$;
```

Característica	Função	Procedimento
Devolve valor	Sim (obrigatoriamente)	Opcional
Uso em SELECT	Sim	Não
Altera dados	Não (em regra)	Sim (pode modificar dados)
Chamado com	Dentro de expressões SQL CALL nome_procedimento(...)	
Propósito comum	Cálculos, validações	Tarefas e operações de negócio

O que é uma vista? Quais as diferenças entre uma vista e uma relação base?
--

O que é uma vista (view) numa base de dados?

Uma **vista** (ou **view**) é uma **tabela virtual** criada a partir de uma ou mais **consultas SELECT** sobre tabelas reais (chamadas **relações base**).

A vista **não armazena dados fisicamente**; apenas **guarda a definição da consulta**. Os dados são atualizados automaticamente sempre que a vista é acedida.

Exemplo de criação de uma vista:

```
CREATE VIEW Vista_Clientes_Activos AS
SELECT nome, email
FROM Clientes
WHERE ativo = TRUE;
```

Característica	Vista (View)	Relação Base (Tabela Real)
Armazena dados	Não – só a consulta	Sim – armazena fisicamente os dados
Atualização dos dados	Reflete os dados atuais das tabelas base	Os dados são geridos diretamente
Definição	Criada com uma instrução SELECT	Criada com CREATE TABLE
Utilidade	Simplificar consultas, ocultar dados sensíveis	Guardar e gerir os dados diretamente
Modificável	Nem sempre (dependendo da complexidade da query)	Sim – totalmente editável

Quando usar uma vista?

- Para simplificar consultas complexas;
- Para restringir o acesso a certas colunas ou linhas;
- Para representar resultados calculados ou filtrados;
- Para separar a lógica da base de dados do acesso do utilizador.

Resumo:

Uma **vista** é uma **representação virtual dos dados**, útil para consulta e segurança, mas **não armazena dados**. Já uma **relação base** é uma **tabela real** que contém os dados diretamente e permite operações de inserção, atualização e eliminação.

Se quiseres, posso mostrar exemplos práticos de uso em diferentes contextos.

Defina vistas e apresente a sua importância.

Definição de Vista (View):

Uma **vista** é uma **tabela virtual** criada a partir de uma instrução SELECT sobre uma ou mais tabelas reais da base de dados (relações base).

Apesar de se comportar como uma tabela, **não armazena dados fisicamente** — apenas **guarda a definição da consulta**.

Importância das Vistas:

1. **Simplicidade:**

Permitem **simplificar consultas complexas**, reutilizando a mesma lógica várias vezes.

2. **Segurança:**

Podem **restringir o acesso** a certos dados, mostrando apenas colunas ou linhas específicas aos utilizadores.

3. **Independência lógica:**

Permitem **abstrair a estrutura interna** das tabelas, protegendo as aplicações de alterações à estrutura da base de dados.

4. **Reutilização de lógica:**

Tornam o código mais limpo e reutilizável, já que várias aplicações ou consultas podem usar a mesma vista.

5. **Representação de dados derivados:**

Facilitam a criação de **dados calculados ou agregados**, como totais, médias ou resultados filtrados.

Resumo:

As **vistas** são ferramentas importantes para **organizar, proteger e simplificar o acesso aos dados** numa base de dados, contribuindo para maior segurança, reutilização e manutenção mais fácil do sistema.

Discuta as vantagens e desvantagens das vistas.

Vantagens das Vistas (Views):

1. **Simplicidade:**

Facilitam o uso de consultas complexas, permitindo que os utilizadores acessem a dados com instruções SELECT simples.

2. **Segurança:**

Podem ser usadas para **restringir o acesso a certos dados**, mostrando apenas as colunas ou linhas autorizadas.

3. **Independência lógica:**

Permitem ocultar a estrutura real das tabelas. Mesmo que as tabelas mudem, a vista pode manter-se igual para os utilizadores.

4. **Reutilização de lógica:**

Evitam repetição de código SQL complexo em várias partes da aplicação, centralizando a lógica de consulta.

5. **Facilidade na manutenção:**

Alterações nos dados reais (tabelas base) refletem-se automaticamente na vista, sem necessidade de alterar o código que usa a vista.

Desvantagens das Vistas:

1. **Performance (desempenho):**

Como os dados não estão armazenados, cada acesso à vista executa a consulta original — o que pode ser lento em vistas muito complexas.

2. **Limitações na atualização:**

Nem todas as vistas podem ser usadas para INSERT, UPDATE ou DELETE. Vistas com JOIN, GROUP BY ou DISTINCT, por exemplo, podem **não ser atualizáveis**.

3. **Dependência de estrutura:**

Se a estrutura das tabelas base mudar significativamente, a vista pode deixar de funcionar e terá de ser atualizada.

4. **Pode ocultar complexidade importante:**

O utilizador pode pensar que está a aceder a dados simples, sem perceber que a vista executa operações complexas e pesadas.

Resumo:

As **vistas** oferecem uma forma poderosa de **organizar, simplificar e proteger o acesso aos dados**, mas devem ser usadas com cuidado, especialmente em sistemas com elevado volume de dados, devido a possíveis impactos no **desempenho e na atualização dos dados**.

Que restrições são necessárias para garantir que a vista é atualizável?

Restrições necessárias para garantir que uma vista é atualizável:

Nem todas as **vistas** (views) permitem fazer operações como INSERT, UPDATE ou DELETE.

Para que uma vista seja **atualizável**, tem de cumprir certas **condições**:

A vista deve:

1. **Basear-se numa única tabela:**

A vista só pode referenciar **uma única relação base**.

→ Vistas que usam JOIN entre várias tabelas **não são atualizáveis**.

2. **Incluir todas as colunas obrigatórias:**

Deve incluir **todas as colunas da tabela base** que **não aceitam valores NULL** e não têm valor por defeito.

→ Sem essas colunas, não é possível inserir novos registos.

3. **Não usar funções agregadas:**

Vistas com SUM, COUNT, AVG, MIN, MAX **não são atualizáveis**, pois apresentam dados derivados, não editáveis diretamente.

4. **Não usar GROUP BY, DISTINCT, UNION ou HAVING:**

Estas cláusulas transformam os dados em conjuntos agregados ou filtrados, impossibilitando a atualização.

5. **Não ter expressões calculadas ou colunas derivadas:**

Exemplo: SELECT nome, salario * 0.23 AS iva FROM Funcionarios → esta vista **não pode ser atualizada**, porque o campo iva não existe na tabela base.

Outras notas importantes:

- Algumas vistas podem ser **tornadas atualizáveis** com **triggers** ou **INSTEAD OF triggers** (dependendo do SGBD).
- Em SGBDs como PostgreSQL ou SQL Server, podes usar **views com WITH CHECK OPTION** para restringir alterações fora dos critérios da vista.

Resumo:

Uma vista é atualizável **apenas quando representa diretamente uma parte simples e não transformada da tabela base**.

Qualquer operação que **modifique ou agregue dados** impede que a vista possa ser usada para alterar diretamente os dados.

O que é a materialização de vistas e as suas vantagens associadas.

O que é a materialização de vistas?

A **materialização de vistas** (ou **views materializadas**) refere-se à criação de uma **vista que armazena fisicamente os dados** resultantes da consulta, ao contrário das **vistas normais**, que apenas guardam a definição da consulta e geram os dados em tempo real.

Ou seja, uma **view materializada** guarda uma **cópia dos dados** e pode ser **atualizada periodicamente** ou manualmente.

Vantagens da materialização de vistas:

1. Melhoria de desempenho:

Como os dados estão **pré-calculados e armazenados**, as consultas sobre a vista são muito mais **rápidas**, especialmente quando envolvem cálculos complexos ou grandes volumes de dados.

2. Redução da carga sobre as tabelas base:

Diminui o número de acessos e cálculos feitos diretamente nas tabelas originais, melhorando o desempenho geral do sistema.

3. Útil em relatórios e estatísticas:

Ideal para **dados agregados ou resumidos**, como totais mensais, médias ou contagens, usados em dashboards ou relatórios.

Desvantagens (para equilíbrio):

- **Armazena dados duplicados**, ocupando espaço.
- Pode ficar **desatualizada**, se não for refrescada com regularidade.
- Pode ser **mais lenta a criar e a manter**, especialmente com dados em constante mudança.

Resumo:

A **materialização de vistas** permite **armazenar os resultados de uma consulta**, melhorando o desempenho em sistemas com dados pesados e consultas repetidas.

É uma solução muito útil para **relatórios, dashboards e análises**, desde que se aceite o custo de manter os dados atualizados.

Descreva como funciona o mecanismo de materialização de vistas.

O **mecanismo de materialização de vistas** consiste em **armazenar fisicamente os dados** resultantes de uma consulta definida numa vista. Esses dados são guardados **numa tabela interna gerida pelo SGBD**.

Resumo:

As vistas materializadas funcionam como **cópias estáticas e otimizadas** de dados, criadas para melhorar o desempenho de consultas pesadas e repetitivas.

Descreva como funciona o mecanismo de resolução de vistas.

O **mecanismo de resolução de vistas** aplica-se às **vistas normais** (não materializadas) e consiste em **substituir a vista pela sua definição original (query) no momento da execução**.

Resumo:

A resolução de vistas permite que **as vistas normais funcionem como atalhos lógicos**, e os dados **não são armazenados**, apenas **calculados em tempo real** com base na sua definição.

Enuncie quais as principais abordagens para elaborar o desenho de uma base de dados com múltiplas vistas de utilizadores

Quando vários utilizadores ou departamentos acedem a diferentes partes da base de dados, é importante planear o desenho da base com **vistas adaptadas às suas necessidades**. As principais abordagens são:

1. Abordagem de cima para baixo (top-down):

- Parte-se de uma **visão geral da organização** (modelo conceptual global).
- Depois, definem-se as **vistas específicas de cada utilizador** com base nessa estrutura.
- Garante **consistência** entre as diferentes vistas e o modelo lógico geral.

Vantagem: ajuda a manter a coerência entre as vistas e facilita a normalização dos dados.

2. Abordagem de baixo para cima (bottom-up):

- Começa-se por **analisar as necessidades individuais** de cada utilizador (as suas vistas).
- Depois, essas vistas são **unificadas e integradas** para construir o modelo global.

Vantagem: adapta-se bem quando os utilizadores têm sistemas independentes ou requisitos distintos.

3. Abordagem mista (ou por integração):

- Combina as duas abordagens anteriores:
 - Define-se uma estrutura base com os dados comuns a todos os utilizadores;
 - Em seguida, são criadas as **vistas específicas** conforme os requisitos de cada grupo.

Vantagem: oferece **equilíbrio entre consistência e flexibilidade**, sendo a mais usada em projetos reais.

4. Abordagem orientada a vistas (view integration):

- Cria-se o sistema **a partir da análise detalhada de cada vista de utilizador**.
- Posteriormente, as vistas são comparadas, reconciliadas e **integradas num esquema conceptual único**.

Vantagem: ideal para ambientes colaborativos, onde os dados vêm de diferentes fontes ou departamentos.

Resumo:

As principais abordagens são:

- **Top-down** (do geral para o específico),
- **Bottom-up** (das vistas para o modelo geral),
- **Mista** (combina as duas anteriores),
- **Orientada a vistas** (construída a partir das perspetivas dos utilizadores).

A escolha da abordagem depende da complexidade do sistema, da organização e do número de utilizadores com diferentes necessidades.

Descreva quais os passos a desenvolver no ciclo de vida de uma aplicação de Bases de Dados.

O ciclo de vida de uma aplicação de bases de dados inclui todas as etapas necessárias para **conceber, desenvolver, implementar e manter** um sistema de base de dados que responda às necessidades dos utilizadores.

1. Estudo e levantamento de requisitos:

- Recolha de informação junto dos utilizadores e da organização.
- Identificação de **o que deve ser armazenado, quem vai usar e que funcionalidades são necessárias**.

2. Projeto conceptual (modelo de alto nível):

- Criação do **modelo Entidade-Relacionamento (ER)** ou equivalente.
- Define as **entidades, atributos e relações**, de forma **independente do SGBD**.
- Resulta num **esquema conceptual**.

3. Projeto lógico:

- Transformação do modelo conceptual para um **modelo relacional** (ou outro, consoante o SGBD).

- Definição de **tabelas, chaves primárias e estrangeiras, tipos de dados e restrições**.
 - Otimização da estrutura através da **normalização**.
-

4. Projeto físico:

- Decisões sobre a **estrutura de armazenamento**, índices, partições, segurança e backups.
 - Adaptação ao **SGBD específico** escolhido (ex: MySQL, SQL Server, PostgreSQL).
-

5. Implementação:

- Criação da base de dados e inserção dos dados iniciais.
 - Programação das **consultas, formulários, interfaces e triggers** necessárias.
-

6. Testes e validação:

- Verificação do correto funcionamento da aplicação.
 - Teste das regras de integridade, desempenho e segurança.
 - Correção de erros.
-

7. Manutenção e evolução:

- Atualização da base de dados consoante novas necessidades.
- Correção de problemas, melhoria de desempenho e gestão de segurança.
- Adaptação a alterações nos dados ou processos da organização.

Resumo:

O ciclo de vida de uma aplicação de base de dados segue uma sequência de etapas:

análise de requisitos → projeto conceptual → projeto lógico → projeto físico → implementação → testes → manutenção, garantindo que o sistema final é eficaz, seguro e adaptado aos utilizadores.

Identifique quais os principais passos a seguir no Desenho Conceptual e Lógico de uma base de dados.

1. Desenho Conceptual

Nesta fase, o objetivo é **modelar os dados de forma abstrata**, com base nos requisitos do sistema, sem se preocupar ainda com tabelas ou tipos de dados.

Passos principais:

1. **Levantamento de requisitos:**
 - Identificar quais os dados importantes para o sistema e como são usados.
 - Entrevistar utilizadores e analisar os processos.
2. **Identificação de entidades e atributos:**
 - Determinar os objetos principais (ex: Cliente, Produto) e as suas características (ex: nome, preço).
3. **Identificação de relacionamentos entre entidades:**
 - Estabelecer as ligações entre entidades (ex: Cliente faz Encomenda).
 - Definir a **cardinalidade** (1:1, 1:N, N:M).
4. **Criação do diagrama Entidade-Relacionamento (ER):**
 - Representar visualmente as entidades, atributos e relacionamentos.
 - Incluir chaves primárias e, se necessário, atributos multivalorados ou compostos.
5. **Revisão e validação com os utilizadores:**
 - Confirmar que o modelo representa corretamente a realidade.

2. Desenho Lógico

Nesta fase, o modelo conceptual é **transformado num modelo relacional**, com tabelas, campos e chaves bem definidas.

Passos principais:

1. **Conversão das entidades em tabelas:**
 - Cada entidade torna-se uma tabela.
 - Atributos tornam-se colunas.
2. **Conversão dos relacionamentos:**
 - Relacionamentos 1:N e N:M são convertidos em **chaves estrangeiras** ou **tabelas auxiliares**, consoante o caso.

3. **Definição das chaves primárias e estrangeiras:**

– Identificar os campos que garantem a unicidade e ligação entre tabelas.

4. **Normalização:**

– Aplicar as formas normais (1FN, 2FN, 3FN) para eliminar redundâncias e anomalias.

– Garantir a integridade dos dados.

5. **Ajustes com base em requisitos específicos:**

– Adicionar restrições, índices ou outras otimizações.

Fase	Objetivo principal	Resultado final
Desenho Conceptual	Representar os dados de forma abstrata e clara	Diagrama ER ou modelo de alto nível
Desenho Lógico	Transformar o modelo conceptual no modelo relacional	Conjunto de tabelas normalizadas

Identifique quais os principais fatores críticos de sucesso no desenho de uma base de dados.

O sucesso de uma base de dados depende da qualidade do seu **desenho inicial**, que deve garantir eficiência, segurança, integridade e adequação às necessidades reais da organização. Eis os principais fatores:

1. Levantamento completo e preciso dos requisitos

- É essencial **compreender bem as necessidades dos utilizadores e da organização.**
- Um levantamento incompleto leva a modelos incorretos ou insuficientes.

2. Modelo conceptual bem estruturado

- Um **bom modelo Entidade-Relacionamento (ER)** facilita todo o processo posterior.
 - Deve refletir com clareza os dados, entidades, atributos e relações reais.
-

3. Normalização adequada

- Aplicar corretamente as formas normais (1FN, 2FN, 3FN...) **evita redundância e anomalias.**
 - Garante integridade e consistência dos dados.
-

4. Definição clara de chaves primárias e estrangeiras

- As **ligações entre tabelas** devem ser bem definidas para garantir **coerência e integridade referencial.**
-

5. Flexibilidade para alterações futuras

- O desenho deve permitir **evolução e adaptação** sem grandes reestruturações.
-

6. Otimização de desempenho

- Considerar o **volume de dados, tipos de consulta e necessidade de índices** desde o início.
 - Um modelo eficiente evita lentidão e sobrecarga.
-

7. Segurança e controlo de acessos

- Deve prever desde o início **quem pode aceder, alterar ou consultar** cada parte da base de dados.
-

8. Documentação clara e completa

- Toda a estrutura e decisões devem estar **bem documentadas** para facilitar a manutenção futura.

Resumo:

Os principais fatores críticos de sucesso incluem:

bom levantamento de requisitos, modelação correta, normalização adequada, integridade referencial, segurança, desempenho e documentação.

Sem estes elementos, o sistema pode tornar-se ineficaz, difícil de manter e vulnerável a erros.

Explique as diferenças entre o processo de especialização e de generalização no contexto da modelação ER.

Diferenças entre Especialização e Generalização no modelo Entidade-Relacionamento (ER):

Tanto a **especialização** como a **generalização** são processos usados para **representar hierarquias entre entidades** no modelo ER, mas **em sentidos opostos**.

1. Especialização

- É o processo de **dividir uma entidade genérica em subconjuntos mais específicos**.
- Cria **subentidades** com atributos e relacionamentos próprios.
- Representa uma **relação "é um tipo de"**.

Exemplo:

A entidade Funcionário pode ser especializada em:

- Engenheiro,
- Administrativo,
- Motorista.

Cada subentidade pode ter atributos diferentes (por exemplo, Engenheiro pode ter "especialidade").

Serve para detalhar diferenças entre subconjuntos de uma entidade mais geral.

2. Generalização

- É o processo inverso: **agrupar duas ou mais entidades específicas numa entidade mais geral**.
- Identifica **semelhanças entre entidades** distintas.

Exemplo:

As entidades Aluno e Professor podem ser generalizadas numa entidade mais geral chamada Pessoa, com atributos comuns como nome e morada.

Serve para evitar repetição de atributos e representar propriedades comuns.

Descreva o que representam os atributos num diagrama ER e dê exemplos de atributos simples, compostos, multi-valor e derivados.

1. Atributo Simples (ou Atômico):

- **Não pode ser dividido** em subpartes.
- Representa um valor único e indivisível.

Exemplo:

- Número do Cartão de Cidadão
 - Nome próprio
-

2. Atributo Composto:

- Pode ser **decomposto em subatributos** mais simples.
- Agrupa informações relacionadas.

Exemplo:

- Endereço → composto por: rua, cidade, código postal
-

3. Atributo Multivalorado:

- Pode conter **mais do que um valor** para a mesma entidade.
- Representado por **dois círculos concêntricos** no diagrama ER.

Exemplo:

- Grau académico → pode ter: Licenciado, Mestre, Doutorado
 - Números de telefone
-

4. Atributo Derivado:

- O seu valor é **calculado a partir de outro atributo**.
- Representado por **linhas tracejadas** no diagrama ER.

Exemplo:

- Idade → derivada da data de nascimento
- Tempo de serviço → derivado da data de admissão

Tipo de Atributo	Característica	Exemplo
Simple	Valor único e indivisível	Número de cartão de cidadão
Composto	Pode ser dividido em subpartes	Endereço → rua, cidade, postal
Multivalorado	Pode ter mais do que um valor	Grau académico
Derivado	Calculado com base noutros atributos	Idade (a partir da data de nascimento)

No contexto do modelo relacional de bases de dados, quais os objetivos da normalização de dados? De que forma o processo de normalização poderá afetar, posteriormente, o desempenho da respetiva implementação?

Objetivos da Normalização de Dados no Modelo Relacional

A **normalização** é um processo utilizado no **desenho lógico de bases de dados** com os seguintes objetivos principais:

1. **Eliminar redundância de dados:**
 - Evita a repetição desnecessária de dados em várias tabelas.
2. **Evitar anomalias de atualização:**
 - Corrige problemas que podem surgir ao **inserir, atualizar ou eliminar dados** (ex: duplicações, inconsistências).
3. **Melhorar a integridade e consistência:**
 - Garante que os dados estão corretamente organizados e que **as dependências entre atributos são respeitadas**.
4. **Organizar os dados de forma lógica e eficiente:**
 - Torna o modelo mais **compreensível e fácil de manter**.

Impacto da Normalização no Desempenho

Embora a normalização tenha muitos benefícios, pode **afetar o desempenho** da base de dados, especialmente em sistemas com muitas consultas:

Vantagens no desempenho:

- Menos dados repetidos → menor ocupação de espaço.
- Evita atualizações desnecessárias → menos operações de escrita.

Desvantagens no desempenho:

- Os dados ficam **distribuídos por várias tabelas**.
- Consultas complexas exigem **mais junções (JOIN)** entre tabelas.
- O número de acessos ao disco pode aumentar, afetando o tempo de resposta.

A **normalização** tem como objetivos principais **evitar redundância, anomalias e melhorar a integridade dos dados**.

Contudo, pode afetar negativamente o desempenho em tempo de execução, devido ao **aumento de junções** entre tabelas, sendo necessário, em alguns casos, **equilibrar entre normalização e desempenho**, usando **desnormalização controlada**.

Enuncie as definições de cada Forma Normal.

Forma Normal Condição Principal

1FN	Valores atômicos; sem atributos multivalorados
2FN	1FN + dependência total da chave primária
3FN	2FN + sem dependências transitivas
BCNF	3FN + em todas as dependências, o determinante é superchave

Tabela Inicial – Não Normalizada

NIF	Nome	Curso	Disciplina	Nota
1234	João Silva	Eng. Informática	BD, Redes	14, 15
5678	Ana Costa	Matemática	Álgebra	18

Problema:

- Atributos **multivalorados**: “Disciplina” e “Nota” têm mais do que um valor por registo.

1FN – Primeira Forma Normal

Regra: Eliminar grupos repetitivos → **valores atômicos** (um valor por célula)

NIF	Nome	Curso	Disciplina	Nota
1234	João Silva	Eng. Informática	BD	14
1234	João Silva	Eng. Informática	Redes	15
5678	Ana Costa	Matemática	Álgebra	18

Correção:

- Separámos os valores múltiplos em linhas diferentes.
- A tabela agora **respeita a 1FN**.

2FN – Segunda Forma Normal

Regra:

- **Atributos não devem depender de parte da chave primária composta (se houver).**

Supondo que a chave primária é (NIF, Disciplina), o atributo Curso depende só do NIF, não da Disciplina → violação da 2FN.

Solução: Separar a tabela em duas:

Tabela Alunos:

NIF	Nome	Curso
1234	João Silva	Eng. Informática
5678	Ana Costa	Matemática

Tabela Notas:

NIF	Disciplina	Nota
1234	BD	14
1234	Redes	15
5678	Álgebra	18

Agora, **todos os atributos dependem totalmente da chave primária** de cada tabela → **2FN atingida**.

3FN – Terceira Forma Normal

Regra:

- Eliminar **dependências transitivas** entre atributos não-chave.

Verificação:

- Em Alunos, o atributo Curso depende do NIF.
- Suponhamos que “**Curso**” tem **atributos próprios** (ex: duração, departamento).
→ Isto sugere **dependência transitiva**: NIF → Curso → Duração.

Solução: criar uma tabela para os cursos:

Tabela Alunos:

NIF	Nome	CursoCod
-----	------	----------

1234	João Silva	INF
------	------------	-----

5678	Ana Costa	MAT
------	-----------	-----

Tabela Cursos:

CursoCod	Curso	Duração
----------	-------	---------

INF	Eng. Informática	3 anos
-----	------------------	--------

MAT	Matemática	3 anos
-----	------------	--------

Tabela Notas:

NIF	Disciplina	Nota
-----	------------	------

1234	BD	14
------	----	----

1234	Redes	15
------	-------	----

5678	Álgebra	18
------	---------	----

Agora cada atributo depende **apenas da chave primária direta**, sem intermediários → **3FN atingida**.

Forma Normal

Objetivo

Ação feita

1FN

Eliminar grupos repetitivos

Separar listas em linhas

2FN

Remover dependências parciais

Criar tabelas separadas para atributos não totalmente dependentes

3FN

Remover dependências transitivas

Isolar atributos que dependem de outros não-chave

Em que circunstâncias devemos usar desnormalização? Dê um exemplo para suportar a sua resposta.

Desnormalização – O que é e quando usar

A **desnormalização** é o processo **intencional de reverter parcialmente a normalização**, ou seja, **introduzir alguma redundância nos dados**, com o objetivo de **melhorar o desempenho** da base de dados.

Circunstâncias em que se usa a desnormalização:

1. **Melhoria de desempenho em consultas complexas:**
 - Quando há **muitas junções (JOIN) entre tabelas**, que tornam as consultas lentas.
2. **Consultas frequentes sobre os mesmos dados agregados:**
 - Se os mesmos resultados forem pedidos repetidamente, pode compensar **armazenar os dados prontos**.
3. **Bases de dados orientadas à leitura (data warehouses):**
 - Onde a **rapidez na consulta é mais importante** do que a atualização constante.
4. **Sistemas com poucas atualizações e muitas leituras:**
 - A redundância é aceitável se os dados mudarem raramente.

Riscos da desnormalização:

- Aumenta a **redundância e o espaço ocupado**.
- Pode introduzir **anomalias de atualização**, se não for bem gerida.

Exemplo de desnormalização:

Situação normalizada:

- Tabela Clientes: contém nome, morada, etc.
- Tabela Faturas: contém cliente_id, data, total.

Para consultar o nome do cliente e total da fatura, é preciso fazer JOIN:

```
SELECT Clientes.nome, Faturas.total
FROM Clientes
JOIN Faturas ON Clientes.id = Faturas.cliente_id;
```

Desnormalização:

Se essa consulta for muito frequente, podemos **guardar o nome do cliente diretamente na tabela Faturas**, evitando o JOIN:

Fatura_ID	Cliente_ID	Nome_Cliente	Total
-----------	------------	--------------	-------

1	123	João Silva	200€
---	-----	------------	------

Agora a consulta é mais rápida, mas o nome do cliente está **duplicado** — se ele mudar, pode ser necessário atualizar em vários sítios.

Resumo:

A **desnormalização** é usada para **otimizar o desempenho** em sistemas com muitas leituras ou consultas pesadas.

Apesar de **violar princípios da normalização**, é **útil quando o tempo de resposta é mais importante do que a consistência automática** dos dados.